Regex Tutorial with SQL & PySpark

Goal

Learn practical regex usage in SQL dialects (PostgreSQL/MySQL/BigQuery/Snowflake/Spark SQL) and PySpark DataFrames: validation, extraction, cleaning, and tokenization.

Sample Data

```
-- Assume a table contacts(id INT, email STRING, phone STRING, note STRING)
-- Example rows:
(1, 'john.doe@example.com', '+91-9876543210', 'Met at #HyderabadTech 2025')
(2, 'alice+promo@sub.domain.io', '9876543210', 'Follow up: invoice-123.45')
(3, 'bad@domain', '12345', 'Tickets: ABC-987 and REF-42')
```

Validation (Email)

PostgreSQL / MySQL 8+ / BigQuery / Snowflake often provide REGEXP_LIKE. Spark SQL supports RLIKE/REGEXP.

```
-- PostgreSQL (using ~ operator)
SELECT id, email, (email ~ '^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$') AS is_valid
FROM contacts;

-- MySQL 8+
SELECT id, email, REGEXP_LIKE(email, '^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\\.[A-Za-z]{2,}$') AS is_valid
FROM contacts;

-- BigQuery
SELECT id, email, REGEXP_CONTAINS(email, r'^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\\.[A-Za-z]{2,}$') AS is_v
FROM `project.dataset.contacts`;

-- Snowflake
SELECT id, email, REGEXP_LIKE(email, '^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$') AS is_valid
FROM contacts;

-- Spark SQL
SELECT id, email, email RLIKE '^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\\.[A-Za-z]{2,}$' AS is_valid
FROM contacts;
```

Extraction (Hashtags, Numbers)

```
-- PostgreSQL: regexp_matches returns set of matches
SELECT id, regexp_matches(note, '#([A-Za-z0-9_]+)\b', 'g') AS hashtags
FROM contacts;

-- BigQuery: REGEXP_EXTRACT
SELECT id, REGEXP_EXTRACT(note, r'#([A-Za-z0-9_]+)\\b') AS first_hashtag
FROM `project.dataset.contacts`;

-- Snowflake / MySQL: REGEXP_SUBSTR / REGEXP_REPLACE / REGEXP_INSTR
SELECT id, REGEXP_SUBSTR(note, '#([A-Za-z0-9_]+)') AS first_hashtag
FROM contacts;

-- Spark SQL
SELECT id, REGEXP_EXTRACT(note, '#([A-Za-z0-9_]+)\\b', 1) AS first_hashtag
FROM contacts;
```

Cleaning (Remove non-digits from phone)

```
-- BigQuery
```

```
SELECT id, REGEXP_REPLACE(phone, r'[^0-9]', '') AS phone_digits
FROM `project.dataset.contacts`;

-- MySQL 8+
SELECT id, REGEXP_REPLACE(phone, '[^0-9]', '') AS phone_digits FROM contacts;

-- Snowflake
SELECT id, REGEXP_REPLACE(phone, '[^0-9]', '') AS phone_digits FROM contacts;

-- Spark SQL
SELECT id, REGEXP_REPLACE(phone, '[^0-9]', '') AS phone_digits FROM contacts;
```

Tokenization (Split words)

```
-- PostgreSQL: regexp_split_to_table
SELECT id, regexp_split_to_table(note, '\\W+') AS token FROM contacts;

-- BigQuery: SPLIT with regex not native; use REGEXP_REPLACE to normalize spaces
SELECT id, SPLIT(REGEXP_REPLACE(note, r'\\W+', ' '), ' ') AS tokens
FROM `project.dataset.contacts`;

-- Spark SQL
SELECT id, SPLIT(REGEXP_REPLACE(note, '[^A-Za-z0-9]+', ' '), ' ') AS tokens FROM contacts;
```

PySpark DataFrame Examples

```
from pyspark.sql import functions as F

# Validate email
contacts_df = contacts_df.withColumn(
    'is_valid_email',
    F.expr("email RLIKE '^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\\.[A-Za-z]{2,}$'")
)

# Extract first hashtag
contacts_df = contacts_df.withColumn(
    'first_hashtag',
    F.regexp_extract('note', r'#([A-Za-z0-9_]+)\\b', 1)
)

# Remove non-digits in phone
contacts_df = contacts_df.withColumn('phone_digits', F.regexp_replace('phone', '[^0-9]', ''))

# Tokenize note into words
contacts_df = contacts_df.withColumn('tokens', F.split(F.regexp_replace('note', '[^A-Za-z0-9]+', ' '),
```

Edge Cases & Tips

Beware of consecutive dots in emails, Unicode domains, and dialect differences (e.g., REGEXP_LIKE vs RLIKE). Anchor patterns to avoid partial matches; use non-greedy quantifiers where appropriate.

Exercises

1) Extract invoice numbers like invoice-123.45 (integer or decimal). 2) Validate Indian mobile numbers with optional country code +91 and separators. 3) Split notes into tokens and count hashtag frequency.

```
-- Hints
-- invoice numbers:  invoice-(\d+(?:\.\d+)?)
-- Indian mobile:    ^\+?91?[- ]?\d{10}$
-- Hashtag tokens:   #([A-Za-z0-9_]+)\b
```

Conclusion

Regex in SQL and PySpark enables powerful, set-based text processing directly in your data systems. Combine patterns with other functions for robust pipelines.